Data Storage & Indexing

Mr. S.K.Patil

Data Storage & Indexing

Storage & File Structure

Overview of Physical Storage Media -

- Several types of data storage exist in most computer systems.
- They are classified by the speed with which data can be accessed, by the cost per unit of data to buy the medium, and by the medium's reliability.
- Available media are –
  - **Cache –**
    - It is fastest and most costly form of storage.
    - Cache memory is relatively small; its use is managed by the computer system hardware.

Data Storage & Indexing

Storage & File Structure

Overview of Physical Storage Media -

- Available media are –
  - **Main Memory –**
    - The storage structure used for data is main memory.
    - The general-purpose machine instructions operate on main memory.
    - Main memory may contain several gigabytes of data on a personal computer, or even hundreds of gigabytes of data in large server systems.
    - It is generally too small (or too expensive) for storing the entire database.
    - The contents of main memory are usually lost if a power failure or system crash occurs.

Data Storage & Indexing

Storage & File Structure

Overview of Physical Storage Media -

- Available media are –
  - **Flash Memory –**
    - Flash memory differs from main memory in that stored data are retained even if power is turned off(or fails).
    - There are two types of flash memory, called NAND and NOR flash.
    - NAND flash has higher storage capacity and is widely used for data storage
    - Flash memory has a lower cost per byte than main memory
    - Flash memory is also increasingly used as a replacement for magnetic disks for storing moderate amounts of data.

Data Storage & Indexing

Storage & File Structure

Overview of Physical Storage Media -

- Available media are –
  - **Magnetic Disk Storage –**
    - The primary medium for the long-term online storage of data is the magnetic disk.
    - Usually, the entire database is stored on magnetic disk.
    - Disk storage survives power failures and system crashes.
    - Disk-storage devices may sometimes fail and destroy data, but such failures usually occur less frequently than system crashes.

Data Storage & Indexing

Storage & File Structure

Overview of Physical Storage Media -

- Available media are –
  - **Optical Storage –**
    - The most popular forms of optical storage are the compact disk (CD) and the digital video disk (DVD).
    - The optical disks used in CD-ROM or DVD-ROM cannot be written, but are supplied with data prerecorded.
    - There are also "record-once" versions called CD-R and DVD-R, which can be written only once; such disks are also called write-once, read-many (WORM) disks.
    - There are also "multiple-write" versions called CD-RW and DVD-RW, which can be written multiple times.

Data Storage & Indexing

Storage & File Structure

Overview of Physical Storage Media -

- Available media are –
  - **Tape Storage –**
    - Tape storage is used primarily for backup and archival data.
    - Although magnetic tape is cheaper than disks, because the tape must be accessed sequentially from the beginning.
    - Tape storage is referred to as sequential-access storage.
    - Tapes have a high capacity (40 to 300 gigabyte tapes are currently available).

Data Storage & Indexing

Storage & File Structure

RAID (Redundant Arrays of Independent Disks) -

- The data-storage requirements have been growing so fast that a large number of disks are needed to store data.
- Having a large number of disks in a system improves the rate at which data can be read or written, if the disks are operated in parallel.
- Several independent reads or writes can also be performed in parallel. Failure of one disk does not lead to loss of data.
- A variety of disk-organization techniques, collectively called redundant arrays of independent disks (RAID), have been proposed to achieve improved performance and reliability.
- RAID systems are used for their higher reliability and higher performance rate, rather than for economic reasons.
- Another reason for RAID use is easier management and operations.

Data Storage & Indexing

Storage & File Structure

RAID (Redundant Arrays of Independent Disks) -

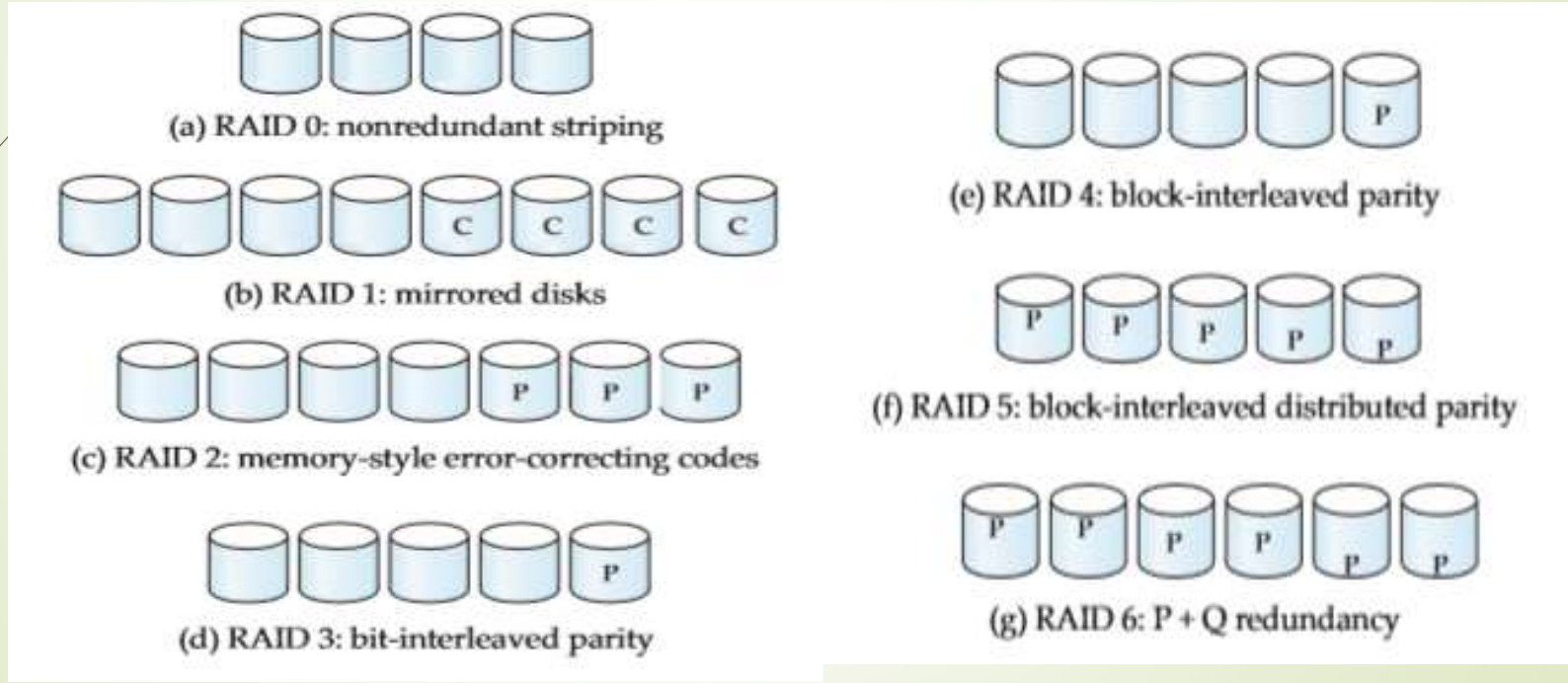
- With multiple disks, we can improve the transfer rate as well by striping data across multiple disks.
- Bit-level striping - splitting the bits of each byte across multiple disks.
- Block-level striping – stripes blocks across multiple disks.
- **RAID Levels –**
  - Mirroring provides high reliability, but it is expensive.
  - Striping provides high data-transfer rates, but does not improve reliability.
  - Various alternative schemes aim to provide redundancy at lower cost by combining disk striping with "parity" bits. The schemes are classified into RAID levels, as in Figure below. (In figure, P – error-correcting bits & C- second copy of data)

Data Storage & Indexing

Storage & File Structure

RAID (Redundant Arrays of Independent Disks) -

- **RAID Levels –**



Data Storage & Indexing

Storage & File Structure

RAID (Redundant Arrays of Independent Disks) -

- **RAID Levels –**
  - RAID level 0 refers to disk arrays with striping at the level of blocks, but without any redundancy.
  - RAID level 1 refers to disk mirroring with block striping. Some vendors use the term RAID level 1+0 or RAID level 10 to refer to mirroring with striping, and use the term RAID level 1 to refer to mirroring without striping.
  - RAID level 2, known as memory-style error-correcting-code (ECC), employs parity bits. Memory systems have long used parity bits for error detection and correction.
  - RAID level 3, bit-interleaved parity, improves level 2 by exploiting the fact that disk controllers can detect whether a sector has been read correctly, so a single parity bit can be used for error correction, as well as for detection.

Data Storage & Indexing

Storage & File Structure

RAID (Redundant Arrays of Independent Disks) -

- **RAID Levels –**
  - RAID level 4, block-interleaved parity, uses block-level striping, like RAID 0, and in addition keeps a parity block on a separate disk for corresponding blocks from N other disks.
  - RAID level 5, block-interleaved distributed parity, improves level 4 by partitioning data and parity among all N+1 disks, instead of storing data in N disks and parity in one disk.
  - RAID level 6, the P + Q redundancy scheme, is much like RAID level 5, but stores extra redundant information to guard against multiple disk failures.

Data Storage & Indexing

Storage & File Structure

File Organization -

- A file is organized logically as a sequence of records.
- These records are mapped on to disk blocks.
- Files are provided as a basic construct in operating systems.
- Each file is logically partitioned into fixed-length storage units called blocks.
- It is the unit of both storage allocation and data transfer.
- Most databases use block sizes of 4 to 8 kilobytes by default.
- Many databases allow specifying the block size at the time of creating a database instance.
- A block may contain several records.
- Usually, record size is smaller than block size.

Data Storage & Indexing

Storage & File Structure

File Organization -

- For large data items such as images the size can vary.
- For accessing data quickly, it is required that one complete record should reside in one block only.
- It should not be divided between one or two blocks.
- In RDBMS, the size of tuples varies in different relations. Thus, we need to structure files in multiple lengths for implementing the records.
- In file organization, there are two possible ways of representing the records.
  - Fixed-length records
  - Variable-length records

Data Storage & Indexing

Storage & File Structure

File Organization -

- **Fixed Length Records -**
  - Fixed-length records means setting a length and storing the records into the file.
  - If the record size exceeds the fixed size, it gets divided into more than one block.
  - Due to fixed size there occurs following wo problems –
    - Partially storing subparts of record in more than one block requires access to all blocks containing the subparts to read or write in it.
    - It is difficult to delete a record in such a file organization, because if the size of existing record is smaller than block size then another record or a part fills up the block.

Data Storage & Indexing

Storage & File Structure

File Organization -

- **Fixed Length Records -**
  - The solution to above problem is to include a certain number of bytes. It is known as File Header.
  - The allocated file header carries a variety of information about the file such as address of first record.
  - The method of insertion and deletion is easy in fixed-length records because the space left or freed by deleted record is exactly similar to the space required to insert new records.
  - But this process fails for storing the records of variable length.

Data Storage & Indexing

Storage & File Structure

File Organization -

- **Variable Length Records -**
  - The variable length records are records that vary in size.
  - It requires the creation of multiple blocks of multiple sizes to store them.
  - These variable length records are kept in the following ways in database system –
    - Storage of multiple record types in a file
    - It is kept as Record Types that enable repeating fields like multisets or array
    - It is kept as Record Types that enable variable lengths either for one field or more.

Data Storage & Indexing

Storage & File Structure

File Organization -

- **Variable Length Records -**
  - In variable length records, there exist following two problems –
    - Defining the way of representing a single record so as to extract the individual attribute easily.
    - Defining the way of storing variable-length records within a block so as to extract that record block quickly.
  - The representation of variable-length record can be divided into two parts -
    - Initial part of record with fixed-length attributes such as numeric values, dates, fixed-length character attributes for storing their value.
    - The data for variable-length attributes such as varchar type is represented in the initial part of record by (offset, length) pair.

Data Storage & Indexing

Storage & File Structure

File Organization -

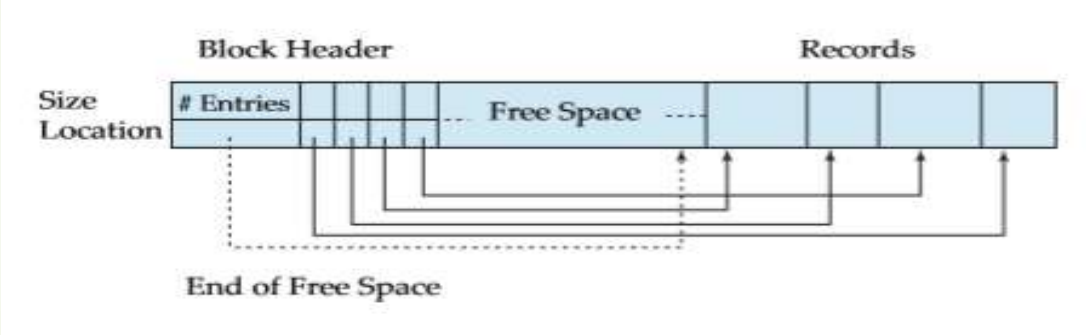
- **Slotted-page structure -**
  - There occurs a problem to store variable-length records within the block.
  - Such a records are organized in a slotted-page structure within the block.
  - In slotted-page structure, a header is present at the starting of each block. This header holds information such as –
    - Number of record entries in the header
    - No free space remaining in the block
    - Array containing information on the location and size of the records

Data Storage & Indexing

Storage & File Structure

File Organization -

- **Slotted-page structure -**



- **Inserting & Deleting Method –**
  - The variable-length records reside in a contiguous manner within block.
  - When new record is to be inserted, it gets place at the end of free space, because free space is contiguous as well.
  - Also, header fills entry with size & location information of newly inserted record

Data Storage & Indexing

Storage & File Structure

File Organization -

- **Slotted-page structure -**
  - **Inserting & Deleting Method –**
    - When existing record is deleted, space is freed & the header entry sets to deleted.
    - Before deleting, it moves the record & occupies it to create the free space.
    - The end of free space gets the update. Then all the free space again sets between first record and final entry.
    - The primary technique of slotted-page structure is that no pointer should directly point the record.
    - Instead it points to header entry which contains information.

Data Storage & Indexing

Storage & File Structure

Organization of Records in File -

- The possible ways of organizing records in files are:
  - **Heap file organization** – Any record can be placed anywhere in the file where there is space for the record. There is no ordering of records. Typically, there is a single file for each relation.
  - **Sequential file organization** - Records are stored in sequential order, according to the value of a "search key" of each record.
  - **Hashing file organization** - A hash function is computed on some attribute of each record. The result of the hash function specifies in which block of the file the record should be placed.
  - **Clustering file organization** – Records of several different relations are stored in the same file. The related records of the different relations are stored on the same block, so that one I/O operation fetches related records from all the relations.

Data Storage & Indexing

Storage & File Structure

Organization of Records in File -

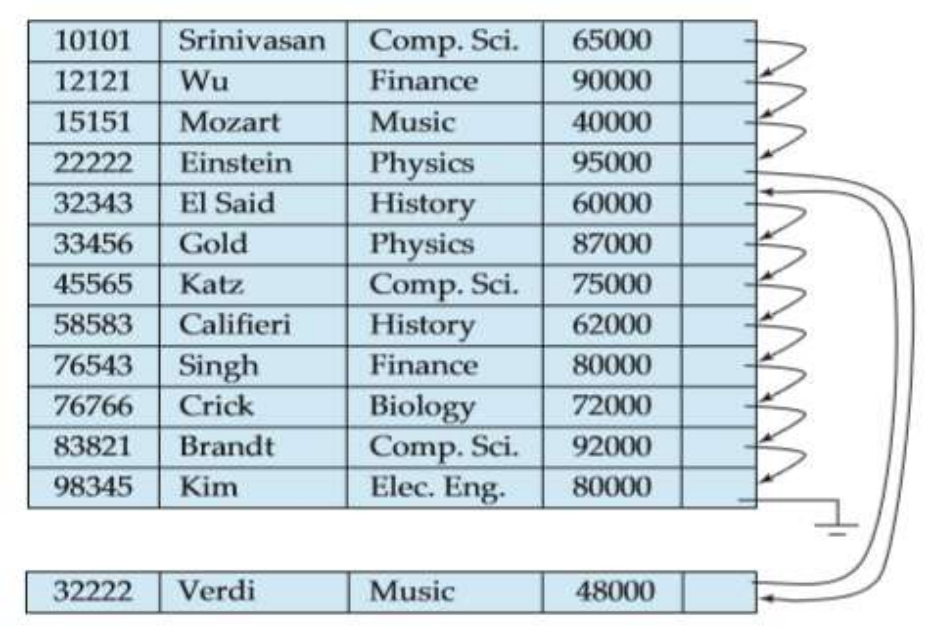
- **Sequential file organization** –
  - A sequential file is designed for efficient processing of records in sorted order based on some search key.
  - A search key is any attribute or set of attributes; it need not be the primary key, or even a superkey.
  - To permit fast retrieval of records in search-key order, we chain together records by pointers.
  - The pointer in each record points to the next record in search-key order.
  - To minimize the number of block accesses in sequential file processing, we store records physically in search-key order, or as close to search-key order as possible.

Data Storage & Indexing

Storage & File Structure

Organization of Records in File -

- **Sequential file organization** –
  - The sequential file organization allows records to be read in sorted order; that can be useful for display purposes.



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

32222	Verdi	Music	48000	

Data Storage & Indexing

Storage & File Structure

Organization of Records in File -

- **Sequential file organization** –
  - It is difficult to maintain physical sequential order as records are inserted and deleted, since it is costly to move many records as a result of a single insertion or deletion.
  - We can manage deletion by using pointer chains.
  - For insertion, must locate the position in the file where the record is to be inserted
    - If there is free space insert there
    - If no free space, insert the record in an overflow block
    - In either case, pointer chain must be updated
  - Need to reorganize file from time to time to restore sequential order.

Data Storage & Indexing

Storage & File Structure

Organization of Records in File -

- **Clustering file organization** –
  - Many relational database systems store each relation in a separate file. Thus, relations can be mapped to a simple file structure.
  - A simple file structure reduces the amount of code needed to implement the system.
  - This simple approach becomes less satisfactory as the size of the database increases.
  - A clustered file organization keeps two or more related tables or records in a single file known as a cluster.
  - These files consist of two or more tables within a single data block, and the mapping attributes, defining the relationships between the tables, are stored only once.

Data Storage & Indexing

Storage & File Structure

Organization of Records in File -

- **Clustering file organization** –
  - It is cheaper to search for and retrieve distinct records from different files as they are integrated and stored in the same cluster.
  - A multitable clustering file organization stores related records of two or more relations in each block.
  - Such a file organization allows to read records that would satisfy the join condition by using one block read.
  - Careful use of multitable clustering can produce significant performance gains in query processing.

Data Storage & Indexing

Storage & File Structure

Organization of Records in File -

- Clustering file organization –

Comp. Sci.	Taylor	100000
45564	Katz	75000
10101	Srinivasan	65000
83821	Brandt	92000
Physics	Watson	70000
33456	Gold	87000

Multitable clustering file structure.

Multitable clustering file structure with pointer chains.

Data Storage & Indexing

Storage & File Structure

Data Dictionary Storage -

- A relational database system needs to maintain data about the relations, such as the schema of the relations.
- In general, such "data about data" is referred to as metadata.
- Relational schemas and other metadata about relations are stored in a structure called the data dictionary or system catalog.
- Types of information that the system must store are :
  - Names of the relations.
  - Names of the attributes of each relation.
  - Domains and lengths of attributes.
  - Names of views defined on the database, and definitions of those views.
  - Integrity constraints (for example, key constraints).

Data Storage & Indexing

Storage & File Structure

Data Dictionary Storage -

- In addition, many systems keep the following data on users of the system:
  - Names of authorized users.
  - Authorization and accounting information about users.
  - Passwords or other information used to authenticate users.
- Further, the database may store statistical and descriptive data about the relations, such as:
  - Number of tuples in each relation.
  - Method of storage for each relation (for example, clustered or non clustered).

Data Storage & Indexing

Storage & File Structure

Data Dictionary Storage -

- The data dictionary may also note the storage organization (sequential, hash, or heap) of relations, and the location where each relation is stored:
  - If relations are stored in operating system files, the dictionary would note the names of the file (or files) containing each relation.
  - If database stores all relations in a single file, dictionary may note the blocks containing records of each relation in a data structure such as a linked list.
- All this metadata information constitutes a miniature database.
- Some database systems store such metadata by using special-purpose data structures and code.
- It is generally preferable to store the data about the database as relations in the database itself.

Data Storage & Indexing

Storage & File Structure

Database Buffer -

- A major goal of the database system is to minimize the number of block transfers between the disk and memory.
- One way to reduce the number of disk accesses is to keep as many blocks as possible in main memory.
- The goal is to maximize the chance that, when a block is accessed, it is already in main memory & thus no disk access is required.
- Since it is not possible to keep all blocks in main memory, we need to manage the allocation of the space available in main memory for the storage of blocks.
- The buffer is that part of main memory which is available for storage of copies of disk blocks.
- The subsystem responsible for the allocation of buffer space is called the buffer manager.

Data Storage & Indexing

Storage & File Structure

Database Buffer -

- **Buffer Manager –**
  - Programs in a database system make requests on the buffer manager when they need a block from disk.
  - If the block is already in the buffer, the buffer manager passes the address of the block in main memory to the requester.
  - If the block is not in the buffer, the buffer manager first allocates space in the buffer for the block, throwing out some other block, if necessary, to make space for the new block.
  - The thrown-out block is written back to disk only if it has been modified since the most recent time that it was written to the disk.
  - Then, the buffer manager reads the requested block from the disk to the buffer, and passes the address of block in main memory to the requester.

Data Storage & Indexing

Storage & File Structure

Database Buffer -

- **Buffer Manager –**
  - The internal actions of the buffer manager are transparent to the programs that issue disk-block requests.
  - The size of the database might be larger than the hardware address space of a machine, so memory addresses are not sufficient to address all disk blocks.
  - To serve the database system well, the buffer manager must use following techniques –
    - **Buffer replacement strategy** - When there is no room left in the buffer, a block must be removed from buffer before a new one can be read in. Most operating systems use a least recently used (LRU) scheme, in which block that was referenced least recently is written back to disk & is removed from the buffer.

Data Storage & Indexing

Storage & File Structure

Database Buffer -

- **Buffer Manager –**
  - The buffer manager use following techniques –
    - **Pinned blocks** - For the database system to be able to recover from crashes, it is necessary to restrict those times when a block may be written back to disk. Most recovery systems require that a block should not be written to disk while an update on the block is in progress. A block that is not allowed to be written back to disk is said to be pinned.
    - **Forced output of blocks** – There are situations in which it is necessary to write back the block to disk, even though the buffer space that it occupies is not needed. This write is called the forced output of a block.

Data Storage & Indexing

Storage & File Structure

Database Buffer -

- **Buffer Replacement Policies –**
  - The goal of a replacement strategy for blocks in the buffer is to minimize accesses to the disk.
  - It is not possible to predict accurately which blocks will be referenced.
  - Therefore, operating systems use the past pattern of block references as a predictor of future references.
  - The assumption generally made is that blocks that have been referenced recently are likely to be referenced again.
  - Therefore, if a block must be replaced, the least recently referenced block is replaced. This approach is called the least recently used (LRU) block-replacement scheme.

Data Storage & Indexing

Storage & File Structure

Database Buffer -

- **Buffer Replacement Policies –**
  - Once the processing of an entire block of tuples in relation is completed, that block is no longer needed in main memory, even though it has been used recently.
  - The buffer manager should be instructed to free the space occupied by a relation block as soon as the final tuple has been processed.
  - This buffer-management strategy is called the toss-immediate strategy.

Data Storage & Indexing

Indexing & hashing

Basic Concepts -

- An index for a file in a database system works in the same way as the index in the textbook.
- The words in the index are in sorted order, making it easy to find the word we want.
- Database system indices play the same role as book indices in libraries.
- For example, to retrieve a student record given an ID, the database system would look up an index to find on which disk block the corresponding record resides, and then fetch the disk block, to get the appropriate student record.
- Keeping a sorted list of students' ID would not work well on very large databases with thousands of students hence keeping the index reduces the search time.

Data Storage & Indexing

Indexing & hashing

Basic Concepts -

- There are two basic kinds of indices:
  - **Ordered indices** - Based on a sorted ordering of the values.
  - **Hash indices -** Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a hash function.
- Each technique must be evaluated on the basis of following factors:
  - **Access types:** The types of access that are supported efficiently. Access types can include finding records with a specified attribute value and finding records whose attribute values fall in a specified range.
  - **Access time:** The time it takes to find a particular data item, or set of items, using the technique in question.

Data Storage & Indexing

Indexing & hashing

Basic Concepts -

- Each technique must be evaluated on the basis of following factors:
  - **Insertion time:** The time it takes to insert a new data item. This value includes the time it takes to find the correct place to insert the new data item, as well as the time it takes to update the index structure.
  - **Deletion time:** The time it takes to delete a data item. This value includes the time it takes to find the item to be deleted, as well as the time it takes to update the index structure.
  - **Space overhead:** The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance.
- An attribute or set of attributes used to look up records in a file is called a **search key.**

Data Storage & Indexing

Indexing & hashing

Ordered Indices -

- To gain fast random access to records in a file, we can use an index structure. Each index structure is associated with a particular search key.
- An ordered index stores the values of the search keys in sorted order, and associates with each search key the records that contain it.
- The records in the indexed file may themselves be stored in some sorted order.
- If the file containing the records is sequentially ordered, a **clustering index** is an index whose search key also defines the sequential order of the file.
- Clustering indices are also called primary indices.
- Indices whose search key specifies an order different from the sequential order of the file are called nonclustering indices, or secondary indices.
- All files are ordered sequentially on some search key. Such files are called index-sequential files.

Data Storage & Indexing

Indexing & hashing

Ordered Indices -

- Following figure shows a sequential file of instructor records. The records are stored in sorted order of instructor ID, which is used as the search key.

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

Data Storage & Indexing

Indexing & hashing

Ordered Indices -

- An index entry, or index record, consists of a search-key value and pointers to one or more records with that value as their search-key value.
- The pointer to a record consists of the identifier of a disk block and an offset within the disk block to identify the record within the block.
- There are two types of ordered indices that we can use:
  - **Dense index:**
    - In this, an index entry appears for every search-key value in the file.
    - In a dense clustering index, the index record contains the search-key value and a pointer to the first data record with that search-key value.
    - The rest of records with same search-key value would be stored sequentially after the first record, because records are sorted on the same search key.

Data Storage & Indexing

Indexing & hashing

Ordered Indices -

- There are two types of ordered indices that we can use:
  - **Dense index:**

Data Storage & Indexing

Indexing & hashing

Ordered Indices -

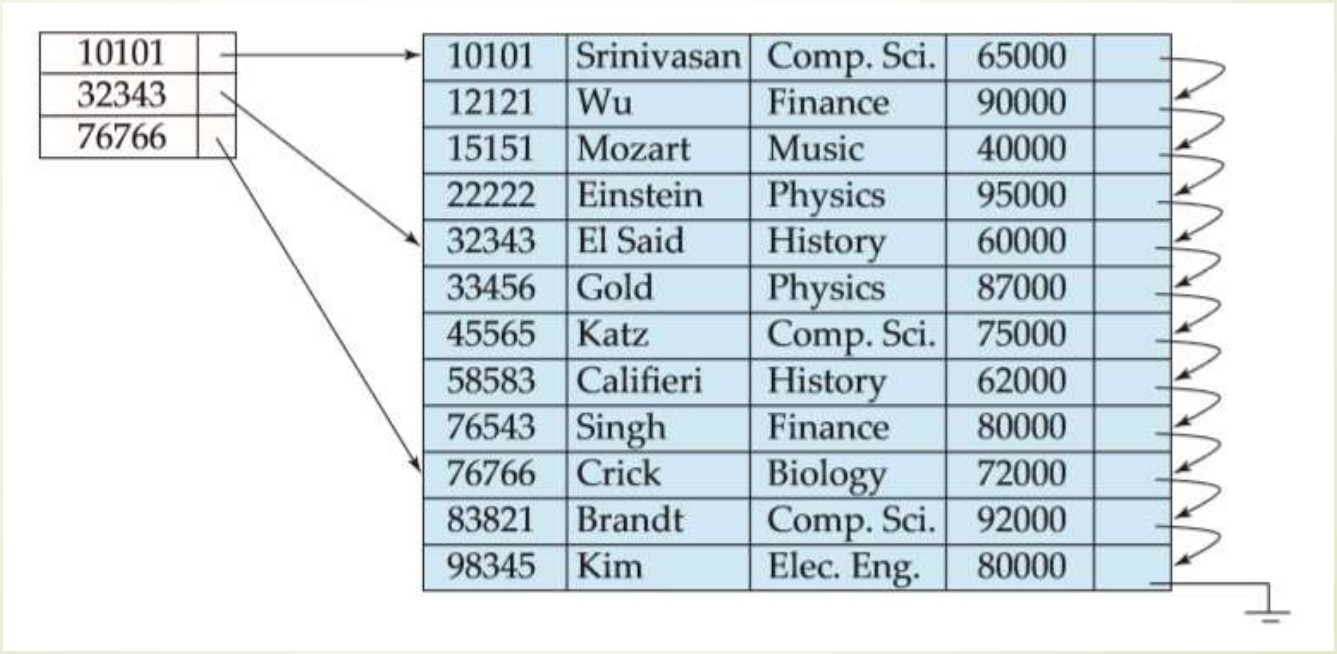
- There are two types of ordered indices that we can use:
  - **Sparse index:**
    - In this, an index entry appears for only some of the search-key values.
    - Sparse indices can be used only if the relation is stored in sorted order of the search key, that is, if the index is a clustering index.
    - Similar to dense, each index entry contains a search-key value and a pointer to the first data record with that search-key value.
    - To locate a record, we find index entry with the largest search-key value that is less than or equal to search-key value for which we are looking.
    - We start at the record pointed to by that index entry, and follow the pointers in the file until we find the desired record.

Data Storage & Indexing

Indexing & hashing

Ordered Indices -

- There are two types of ordered indices that we can use:
  - **Sparse index:**



Data Storage & Indexing

Indexing & hashing

Ordered Indices -

- Suppose, we are looking for the record of instructor with ID "22222".
- Using the dense index, we follow the pointer directly to the desired record. Since ID is a primary key, there exists only one such record & the search is complete.
- If we are using the sparse index, we do not find an index entry for "22222". Since the last entry (in numerical order) before "22222" is "10101", we follow that pointer. We then read the instructor file in sequential order until we find the desired record.
- It is generally faster to locate a record using dense index rather than a sparse index.
- However, sparse indices have advantages over dense indices is that they require less space and they impose less maintenance overhead for insertions and deletions.

Data Storage & Indexing

Indexing & hashing

Ordered Indices -

- Suppose, we are looking for the record of instructor with ID "22222".
- Using the dense index, we follow the pointer directly to the desired record. Since ID is a primary key, there exists only one such record & the search is complete.
- If we are using the sparse index, we do not find an index entry for "22222". Since the last entry (in numerical order) before "22222" is "10101", we follow that pointer. We then read the instructor file in sequential order until we find the desired record.
- It is generally faster to locate a record using dense index rather than a sparse index.
- However, sparse indices have advantages over dense indices is that they require less space and they impose less maintenance overhead for insertions and deletions.

Data Storage & Indexing

Indexing & hashing

Multilevel Indices -

- In RDBSM, indexes are essential data structures that allow faster data retrieval by reducing the number of disk accesses required to retrieve data.
- Traditional indexes can become insufficient as the database size grows.
- Multilevel indexes provide a solution to this problem by dividing the index into smaller, manageable pieces.
- Indexing helps to optimize the performance of a database. It minimizes the number of disk accesses required when a query is processed. It is used to quickly locate and access the data in database.
- There are two things used in indexing – search key & pointer.
- Search key contains the copy of primary key of the table.
- Pointer contains set of pointers holding address of the disk block.

Data Storage & Indexing

Indexing & hashing

Multilevel Indices -

- **Need for Multilevel Indexes –**
  - As the size of database increases, the size of the index also increases. But, storing a single-level index in main memory may not be practical due to its large size, which require multiple disk accesses.
  - To solve this problem multilevel indexing is used.
  - This technique breaks down the main block into smaller blocks. Each smaller block can be stored in a single block.
  - The outer block is further divided into inner blocks, each of which is associated with a data block.
  - This approach reduces overhead and makes it easier to store the index in main memory.

Data Storage & Indexing

Indexing & hashing

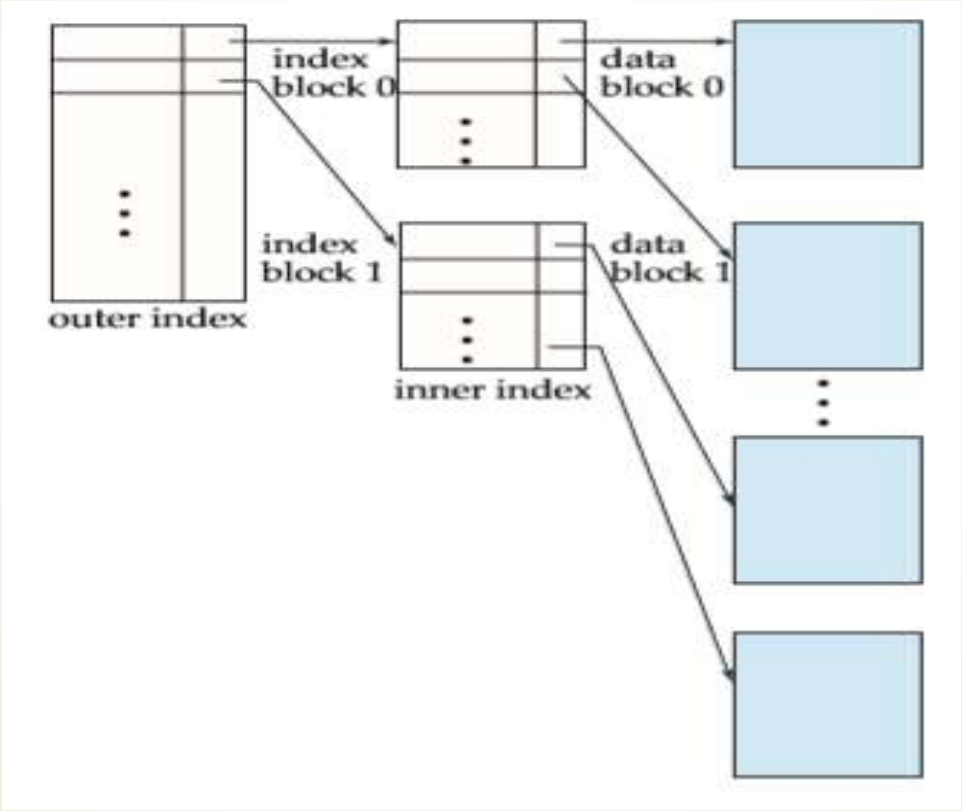
Multilevel Indices -

- Multilevel Indexes refer to hierarchical structure of indexes.
- Here, each level of the index provides a more detailed reference to the data.
- It allows faster data retrieval, reduces disk access and improves query performance.
- Multilevel indexes are essential in large databases where traditional indexes are not efficient.
- A multilevel index can be created for any first-level index (primary, secondary or clustering) that has more than one disk block.
- Its like a search tree, but adding or removing new index entries is challenging because every level of the index is an ordered file.

Data Storage & Indexing

Indexing & hashing

Multilevel Indices -



Data Storage & Indexing

Indexing & hashing

Hashing-

- Hashing is a technique to quickly locate a data record in a database irrespective of the size of the database.
- For larger databases containing thousands and millions of records, the indexing technique becomes very inefficient because searching a specific record through indexing will consume more time.
- This doesn't align with the goals of DBMS, especially when performance and data retrieval time are minimized. So, to counter this problem hashing technique is used.

Data Storage & Indexing

Indexing & hashing

What is Hashing? -

- The hashing technique utilizes an auxiliary hash table to store the data records using a hash function.
- There are 2 key components in hashing:
  - **Hash Table :**
    - A hash table is an array or data structure and its size is determined by the total volume of data records present in the database.
    - Each memory location in a hash table is called a 'bucket' or hash indices.
    - It stores a data record's exact location and can be accessed through a hash function.

Data Storage & Indexing

Indexing & hashing

What is Hashing? -

- There are 2 key components in hashing:
  - **Bucket :**
    - A bucket is a memory location (index) in the hash table that stores the data record.
    - These buckets generally store a disk block which further stores multiple records. It is also known as the hash index.
  - **Hash Function :**
    - A hash function is a mathematical equation or algorithm that takes one data record's primary key as input and computes the hash index as output.

Data Storage & Indexing

Indexing & hashing

Hash Function -

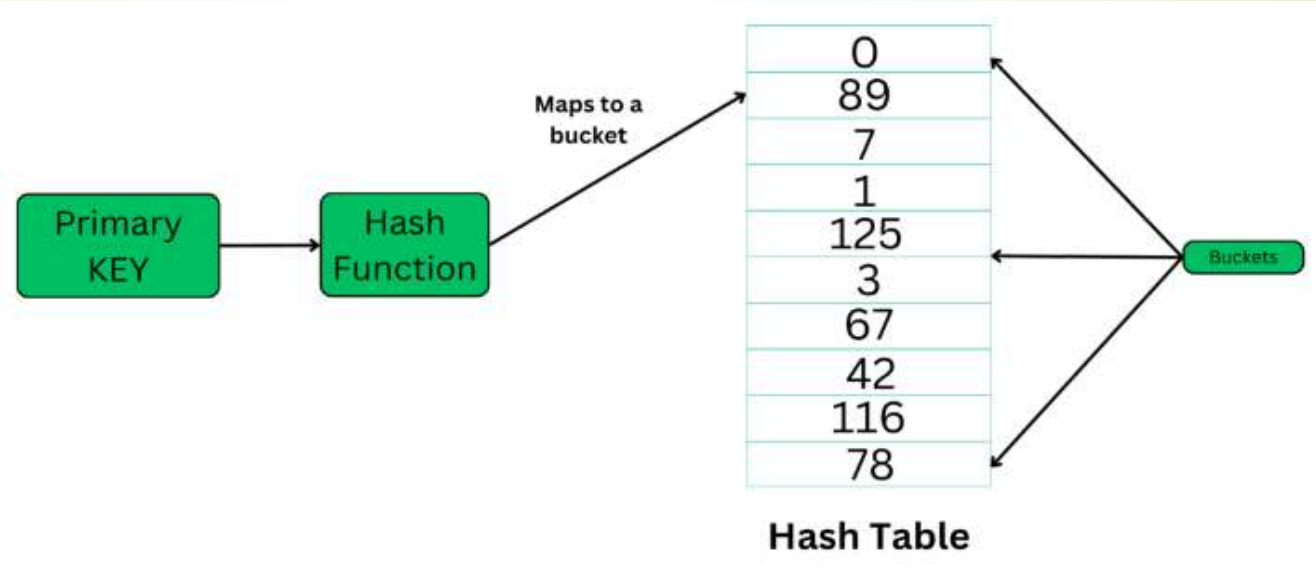
- It is a mathematical algorithm that computes the index or the location where the current data record is to be stored in the hash table so that it can be accessed efficiently later.
- This hash function is the most crucial component that determines the speed of fetching data.
- **Working -** The hash function generates a hash index through the primary key of the data record. Now, there are 2 possibilities:
  - The hash index generated isn't already occupied by any other value. So, the address of the data record will be stored here.
  - The hash index generated is already occupied by some other value. This is called collision so to counter this, a collision resolution technique will be applied.

Data Storage & Indexing

Indexing & hashing

Hash Function -

- Now whenever we query a specific record, the hash function will be applied and returns the data record comparatively faster than indexing because we can directly reach the exact location of the data record through the hash function rather than searching through indices one by one.



Data Storage & Indexing

Indexing & hashing

Types of Hashing -

- There are two primary hashing techniques –
  - Static Hashing
  - Dynamic Hashing
- **Static Hashing –**
  - In static hashing, the hash function always generates the same bucket's address.
  - Example - If we have a data record for employee_id = 106, hash function is mod-5 i.e. H(x) % 5, where x = id. Then the operation will take place like this:

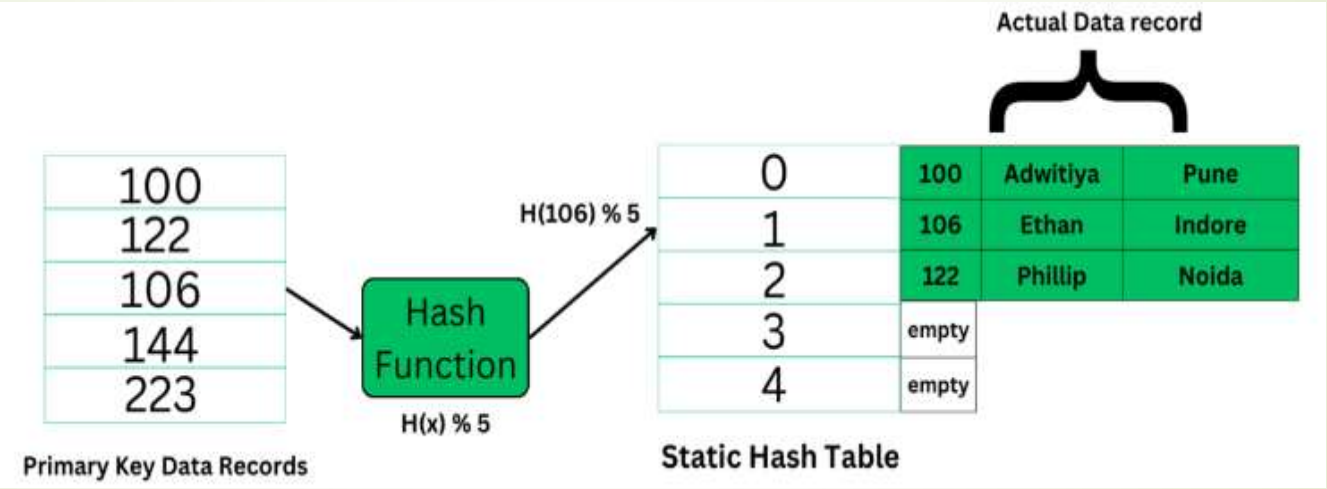
```
H(106) % 5 = 1.
This indicates that the data record should be placed or searched in the 1st
bucket (or 1st hash index) in the hash table.
```

Data Storage & Indexing

Indexing & hashing

Types of Hashing -

- **Static Hashing –**



- The primary key is used as the input to the hash function and the hash function generates the output as the hash index (bucket's address) which contains the address of the actual data record on the disk block.

Data Storage & Indexing

Indexing & hashing

Types of Hashing -

- **Static Hashing –**
  - Static hashing has following properties –
    - **Data Buckets** - The number of buckets in memory remains constant. The size of the hash table is decided initially.
    - **Hash Function** - It uses the simplest hash function to map the data records to its appropriate bucket. It is generally modulo-hash function.
    - **Efficient for known data size**: It's very efficient in terms when we know the data size and its distribution in the database.

Data Storage & Indexing

Indexing & hashing

Types of Hashing -

- **Static Hashing –**
  - It is inefficient and inaccurate when data size dynamically varies because we have limited space & hash function always generates the same value for every specific input.
  - When data size fluctuates very often it's not at all useful because collision will keep happening and it will result in problems like – bucket skew, insufficient buckets etc.
  - To resolve this problem of bucket overflow, techniques such as – chaining and open addressing are used.

Data Storage & Indexing

Indexing & hashing

Types of Hashing -

- **Static Hashing –**
  - **Chaining –**
    - It is a mechanism in which hash table is implemented using an array of type nodes, where each bucket is of node type and can contain a long chain of linked lists to store the data records.
    - Even if a hash function generates the same value for any data record it can still be stored in a bucket by adding a new node.
    - This will gives the problem of bucket skew that is, if hash function keeps generating the same value again and again then the hashing will become inefficient as the remaining data buckets will stay unoccupied or store minimal data.

Data Storage & Indexing

Indexing & hashing

Types of Hashing -

- **Static Hashing –**
  - **Open addressing/ Closed Hashing –**
    - This is also called as closed hashing.
    - This aims to solve the problem of collision by looking out for the next empty slot available which can store data.
    - It uses techniques like linear probing, quadratic probing, double hashing, etc.

Data Storage & Indexing

Indexing & hashing

Types of Hashing -

- **Dynamic Hashing –**
  - Dynamic hashing is also known as extendible hashing.
  - It used to handle database that frequently changes data sets.
  - This method offers us a way to add and remove data buckets on demand dynamically.
  - This way as the number of data records varies, the buckets will also grow and shrink in size periodically whenever a change is made.

Data Storage & Indexing

Indexing & hashing

Types of Hashing -

- **Dynamic Hashing –**
  - **Properties** –
    - The buckets will vary in size dynamically periodically as changes are made offering more flexibility in making any change.
    - Dynamic Hashing helps in improving overall performance by minimizing or completely preventing collisions.
    - It has the following major components: Data bucket, Flexible hash function, and directories
    - A flexible hash function means that it will generate more dynamic values and will keep changing periodically asserting to the requirements of the database.

Data Storage & Indexing

Indexing & hashing

Types of Hashing -

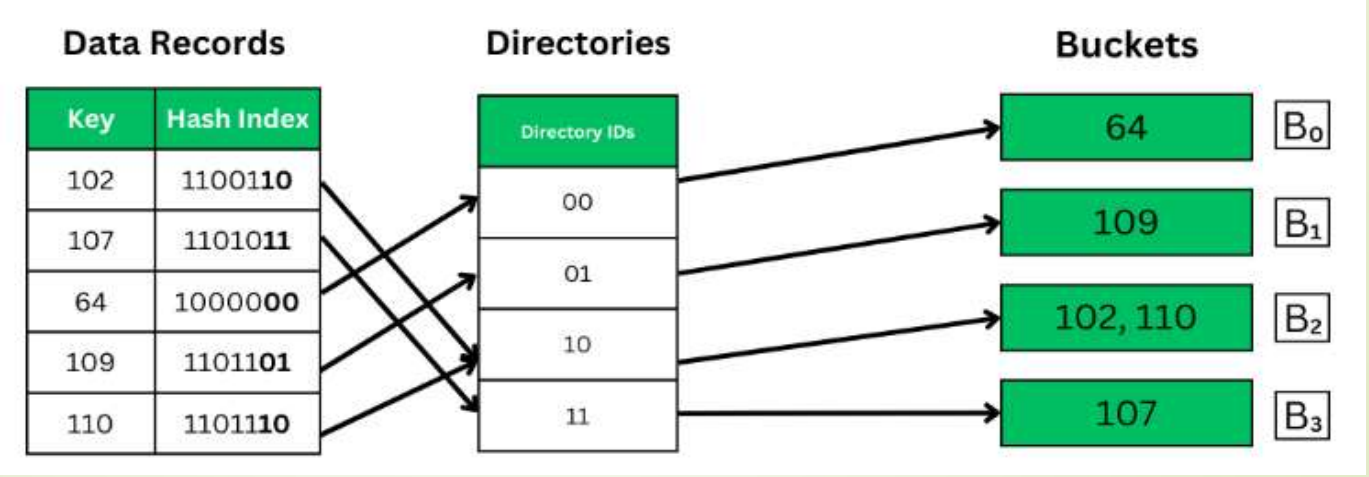
- **Dynamic Hashing –**
  - **Properties** –
    - Directories are containers that store the pointer to buckets. If bucket overflow or bucket skew-like problems happen to occur, then bucket splitting is done to maintain efficient retrieval time of data records. Each directory will have a directory id.
    - **Global Depth**: It is defined as the number of bits in each directory id. The more the number of records, the more bits are there.

Data Storage & Indexing

Indexing & hashing

Types of Hashing -

- **Dynamic Hashing –**
  - **Working of Dynamic Hashing** –
    - Example - If global depth: k = 2, the keys will be mapped accordingly to the hash index. K bits starting from LSB will be taken to map a key to the buckets. The following 4 possibilities: 00, 11, 10, 01.



Data Storage & Indexing

Indexing & hashing

Types of Hashing -

- **Dynamic Hashing –**
  - **Working of Dynamic Hashing** –
    - In the above image, the k bits from LSBs are taken in the hash index to map to their appropriate buckets through directory IDs.
    - The hash indices point to the directories, and the k bits are taken from the directories' IDs and then mapped to the buckets.
    - Each bucket holds the value corresponding to the IDs converted in binary.